

ПРЕГОВОР:

Видове модификатори за достъп:

Modifier	Description
<code>public</code>	The code is accessible for all classes
<code>private</code>	The code is only accessible within the same class
<code>protected</code>	The code is accessible within the same class, or in a class that is inherited from that class. You will learn more about inheritance in a later chapter
<code>internal</code>	The code is only accessible within its own assembly, but not from another assembly. You will learn more about this in a later chapter

Автоматични свойства:

```
{  
    public string Name // property  
    { get; set; }  
}
```

По-стар и подробен синтаксис, в който може да се вложи логика за валидация:

```
class Person  
{  
    private string name; // field  
  
    public string Name // property  
    {  
        get { return name; } // get method  
        set { name = value; } // set method  
    }  
}
```

Термин	Значение
Поле	Променлива в класа, която съхранява данни
Свойство	Позволява контрол върху полето чрез get и set
Конструктор	Метод, който задава начални стойности на обект
this	Означава "текущия обект" – използва се, когато имената съвпадат

КОНСТРУКТОР И this.

Откъде може да учите: https://www.w3schools.com/cs/cs_class_members.php

Задача: Когато създадем клас Car, IntelliSense веднага ни подсказва нов метод, който носи същото име Car() – скобите подсказват метод. Този специален метод, чието име съвпада с името на класа, в който се намира, се нарича конструктор.

```

0 references
internal class Car
{
    public Car() { }
}
метод

```

The image shows a code editor snippet for a C# class named 'Car'. The class is defined as 'internal class Car' followed by an opening curly brace. Inside the class, there is a public method 'Car()' followed by a closing curly brace. A red box highlights the 'Car()' part, and a red arrow points to it from the word 'метод' (method) written below. Above the class definition, it says '0 references'.

Вместо, както досега, всеки път при създаване на нов обект да пишем неговото **име_на_обект.свойство** = стойност, и така за всеки нов обект, може да направим това уточняване веднъж и после да задаваме стойностите директно между скобите още при създаването на обекта.

```
internal class Car
{
    public string Model;
    public string Color;
    public int Year;
    2 references
    public Car(string m, string c, int y)
    {
        Model = m;
        Color = c;
        Year = y;
    }
}
```

Вече може в Main() да подаваме директно стойности, още при самото създаване с new:

```
Car car1 = new Car("Shkoda", "blue", 1990);
```

При public с главна буква или private с малка буква (за защита):

```
private string model;  
private string color;  
private int year;
```

```
public string Model;  
public string Color;  
public int Year;
```

1 reference

```
public Car(string m, string c, int y)  
{  
    model = m;  
    color = c;  
    year = y;  
}
```

1 reference

```
public void PrintInfo()  
{  
    Console.WriteLine($"Колата е {model}, {color}, {year}");  
}
```

Да, обаче, докато пишем новия обект в Main(), нямаме особена яснота какво точно означават тези променливи букви като описание – много по-добре би било, ако ги напишем с пълните им думи:

```
Car car1 = new Car("Shkoda", "blue", 1990);  
car1.PrintInfo();
```

 Car.Car(string m, string c, int y)

Как публичният (достъпен) конструктор да използва private, ако имената съвпадат за по-голяма яснота в Main()?

```
private string model;  
private string color;  
private int year;
```

не хваща private,

```
public string Model;  
public string Color;  
public int Year;
```

присвоява себе си

1 reference

```
public Car(string model, string color, int year)  
{
```

```
    model = model;  
    color = color;  
    year = year;  
}
```

 (parameter) string model

'model' is not null here.

CS1717: Assignment made to same variable; did you mean to assign something else?

Точно в тези случаи използваме ключова дума **this**.

```
private string model;
private string color;
private int year;
```

позна полето за защита

```
public string Model;
public string Color;
public int Year;
```

1 reference

```
public Car(string model, string color, int year)
{
    this.model = model;
    this.color = color;
    this.year = year;
}
```

 (field) string Car.model
'model' may be null here.

Сега вече, ако задържим за описанието какви стойности иска метода, прочитаме:

```
Car car1 = new Car("Shkoda", "blue", 1990);
car1.PrintInfo();
```



Car.Car(string model, string color, int year)


```
private string model;
private string color;
private int year;
```

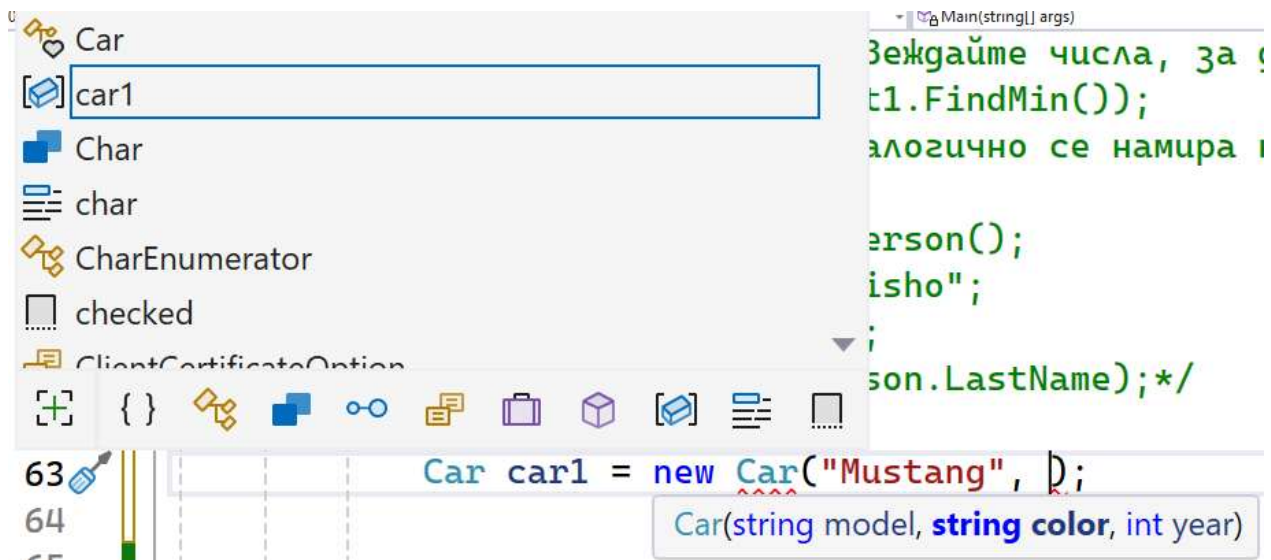
0 references

```
public Car(string model, string color, int year)
{
    this.model = model;
}
```

Едното **this.model** хваща полето (field), а другото модел прихваща променливата **model** от скобите:

```
public Car(string model, string color, int year)
{
    this.model = model;
    this.color = color;
    this.year = year;
}
```

Така си правим конструкция за защита и с готово присвояване – конструктор.

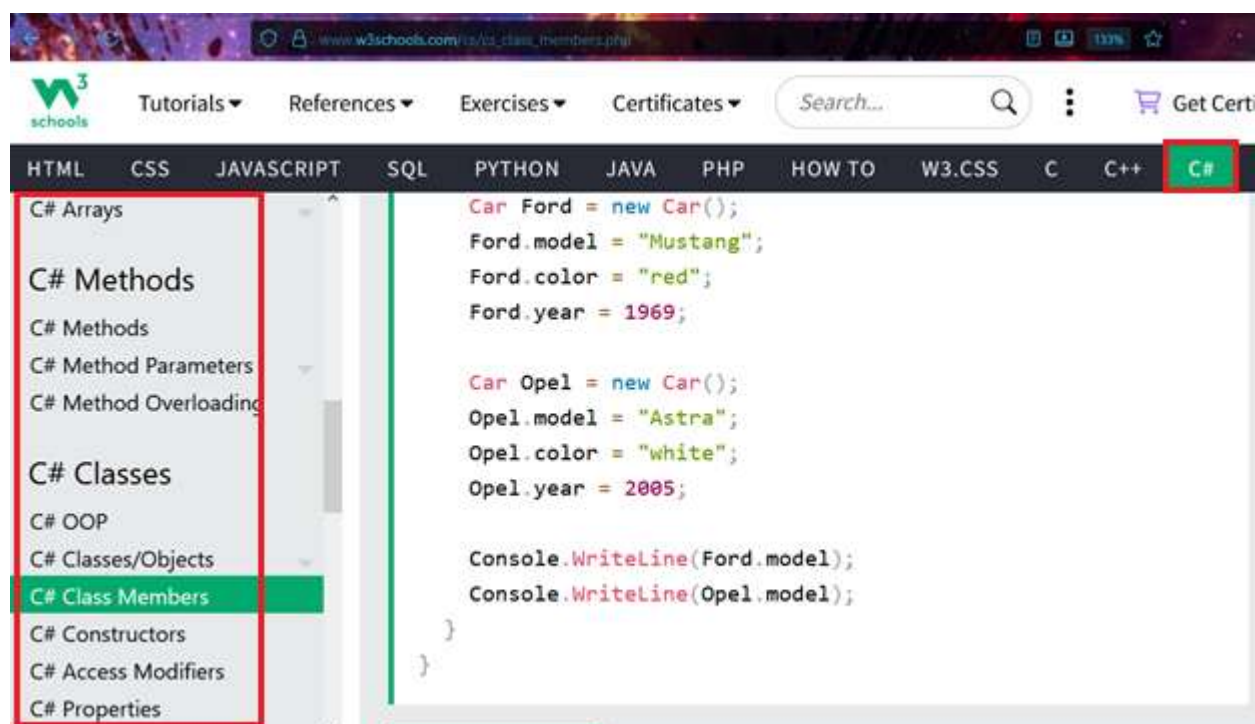


```
public string Model;  
public string Color;  
public int Year;
```

2 references

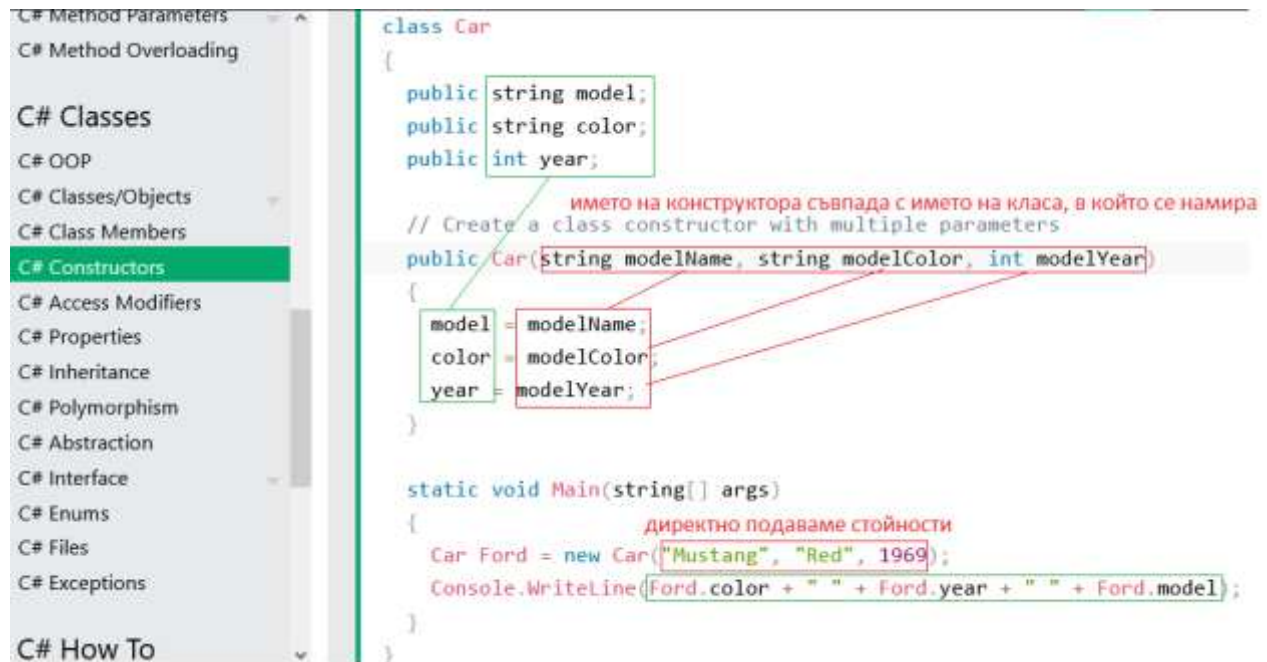
```
public Car(string x, string y, int z)  
{  
    Model = x;  
    Color = y;  
    Year = z;  
}
```

Ако използваме частно поле private field, ще се хвърли грешка `model = model;`; затова има уточняваща ключова дума `this`. – това е чест похват в езиките за програмиране, т.к.



Конструкторът е специален метод, който носи името на класа, в който се намира. Благодарение на него се спестява време и писане при подаване на стойности за нов обект. Ако има конструктор,

можем да подаваме стойностите директно при създаването на обекта между скобите, като ги изреждаме със запетаи.

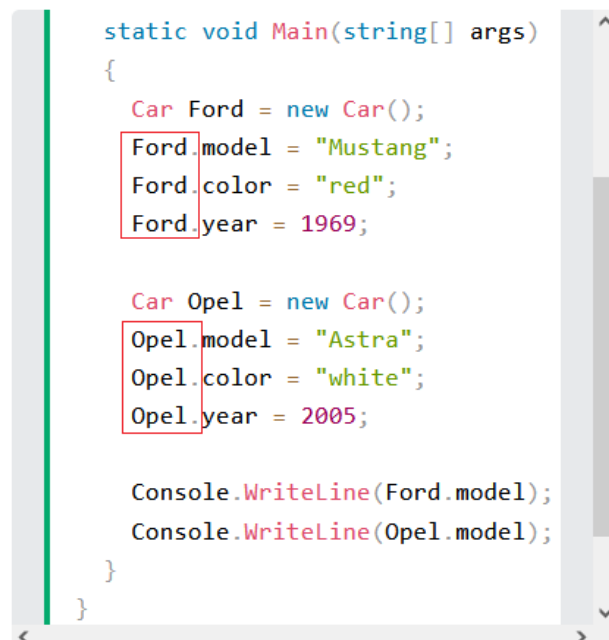


Ние задаваме подробния запис веднъж:

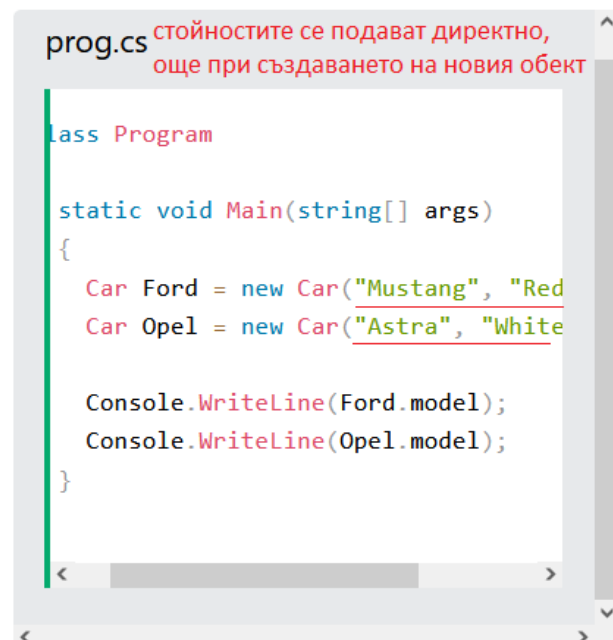
- * свързваме private field – input value
- * public property – input value

в тялото на конструктора и след това, в `Main()` остава само да изреждаме кои точно стойности още при създаването на обекта.

Without constructor:



With constructor:



Т.е. без конструктор всеки път трябва да пишем **Обект.променлива =** , докато със конструктор, това се прави само веднъж (в самия конструктор) и тогава за обекта остава само подаване на стойности между скобите.

Try it Yourself »

```
class Car
{
    public string Model;
    public string Color;
    public int Year;

    // Create a class constructor with multiple parameters
    public Car(string modelName, string modelColor, int
    modelYear)
    {
        Model = modelName;
        Color = modelColor;
        Year = modelYear;
    }

    static void Main(string[] args)
    {
        Car Ford = new Car("Mustang", "Red", 1969);
        Console.WriteLine(Ford.color + " " + Ford.year + " " +
        Ford.model);
    }
}
```

конструктор

Red 1969 Mustang

